LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF330805)
SEMESTER III



TUGAS PRAKTIKUM ACARA 19

NAMA:

RAJATA PRINGGIT BINTANG BIHAR

GOLONGAN:

A

NIM:

E41250264

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

PENDAHULUAN

1. Latar Belakang

Pada Acara 17 dan 18, API Sistem Informasi Akademik sudah memakai arsitektur MVC dengan format response JSON yang seragam. Namun semua endpoint masih terbuka untuk siapa saja. Acara 19 menyiapkan fondasi autentikasi: tabel users, Model User, dan AuthController dengan aksi register() yang menyimpan kata sandi dalam bentuk hash. Pengujiannya dilakukan memakai Postman yang diorganisasi dalam satu Collection dengan Environment, agar alamat server dapat diganti tanpa mengubah request.

2. Tujuan

1. Membuat Postman Collection "API Sistem Informasi Akademik (MVC)" dengan Environment Local dan variabel base_url.
2. Membuat tabel users dan Model User (app/models/User.php).
3. Membuat AuthController dengan aksi register() yang menyimpan kata sandi memakai password_hash().
4. Mendaftarkan resource register pada Router.
5. Menguji endpoint register dan mendokumentasikan contoh response sukses dan gagal.
6. Mengerjakan tugas mandiri: validasi email dengan filter_var() dan Environment Staging.

3. Dasar Teori

API Testing adalah pengujian terhadap endpoint untuk memastikan setiap request menghasilkan response yang sesuai spesifikasi, meliputi validitas endpoint, metode HTTP, parameter, struktur JSON, kode status, dan mekanisme autentikasi. Postman adalah aplikasi untuk mengirim HTTP request, mengatur header dan body, menyimpan request dalam Collection, serta mendokumentasikan API. Variabel pada Environment (misalnya {{base_url}}) memungkinkan Collection yang sama dipakai untuk server yang berbeda.

Pada REST API terdapat beberapa skema autentikasi yang umum, seperti pada Tabel 1. Untuk aplikasi dengan satu jenis klien yang dikendalikan sendiri, Bearer Token yang stateless sudah memadai dan lebih ringan daripada OAuth.

Tabel 1. Skema autentikasi pada REST API

Skema	**Cara kerja**
Basic Authentication	Username dan password di-encode Base64 dan dikirim pada setiap request.
API Key	Token unik untuk klien, dikirim lewat header atau parameter request.
Bearer Token (JWT)	Token diperoleh setelah login berhasil, dikirim pada header Authorization di request berikutnya.

Kata sandi tidak boleh disimpan sebagai teks biasa. Menurut PHP Manual, password_hash() dengan algoritma PASSWORD_DEFAULT direkomendasikan untuk membuat hash yang aman, dan password_verify() dipakai untuk mencocokkan kata sandi saat login (akan dipakai

pada Acara 20). Fungsi filter_var() dengan FILTER_VALIDATE_EMAIL dipakai untuk memeriksa format email.

HASIL PRAKTIK

1. Implementasi

a. Perubahan File

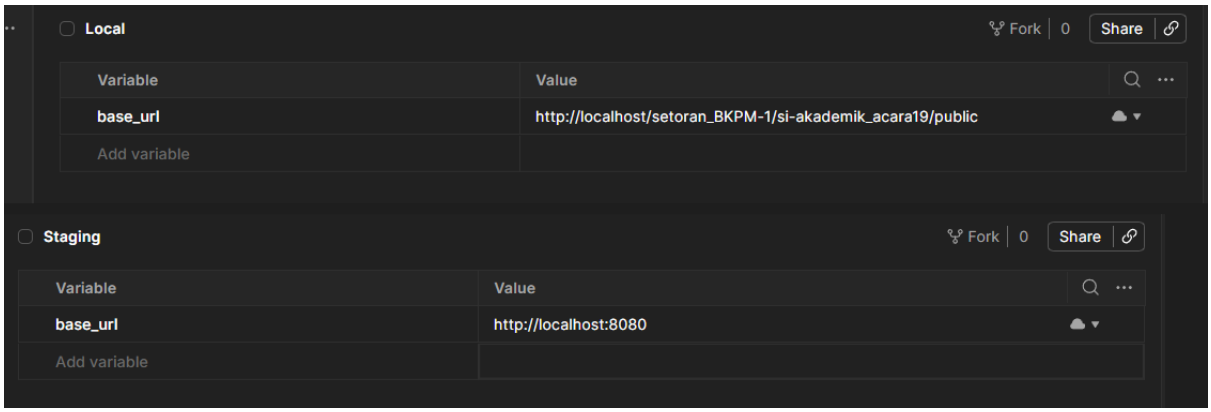
Acara 19 menambahkan resource baru (autentikasi) ke arsitektur MVC yang sudah berjalan. Model, Controller, dan Router untuk mahasiswa dan buku tidak diubah.

Tabel 2. Daftar perubahan file

File	Status	Keterangan
app/models/User.php	Baru	Model User: create(), findByUsername(), validasi email
app/controllers/AuthController.php	Baru	Aksi register()
app/core/Router.php	Diubah	Resource register (hanya POST)
db_workshop.sql	Diubah	Ditambah tabel users
db_acara19_users.sql	Baru	Hanya tabel users, untuk database yang sudah ada
postman/*.json	Baru	Collection, Environment Local, Environment Staging

b. Langkah 1 - Postman Collection dan Environment Local

Dibuat Collection bernama "API Sistem Informasi Akademik (MVC)" berisi dua folder: Auth (request register) dan Mahasiswa (request dari Acara 17-18). Seluruh URL memakai {{base_url}} tanpa akhiran .php karena diarahkan oleh Front Controller dan Router. Environment Local berisi variabel base_url dengan nilai yang menunjuk ke folder public proyek (Gambar 1).

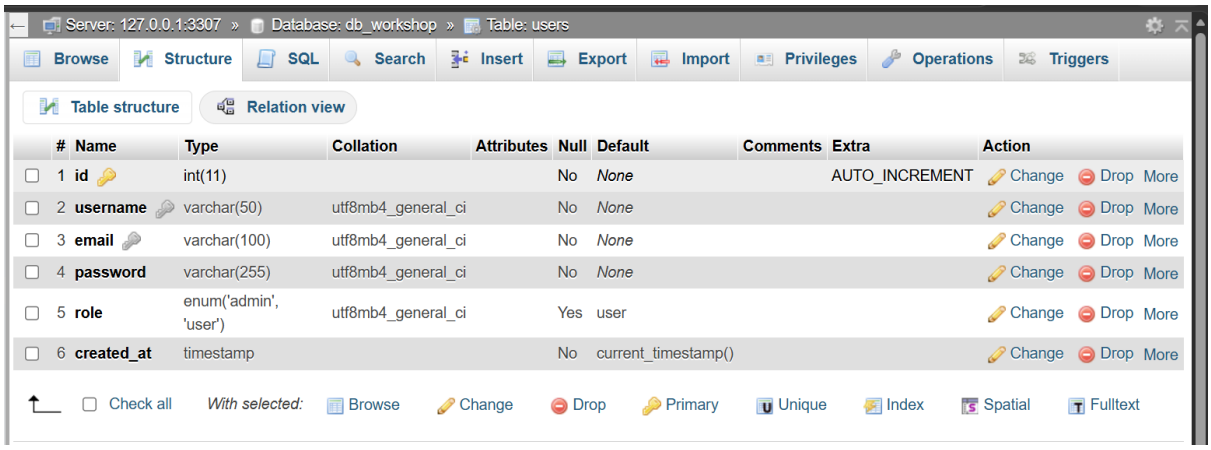


Gambar 1. Environment Local dan Staging (variabel base_url)

c. Langkah 2 - Tabel users

Kolom username dan email bersifat UNIQUE, password VARCHAR(255) agar cukup menampung hash, dan role bernilai admin atau user (default user). Hasilnya terlihat pada Gambar 2.

```
CREATE TABLE IF NOT EXISTS users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    role ENUM('admin','user') DEFAULT 'user',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```



Gambar 2. Struktur tabel users

d. Langkah 3 - Model User

Model User bertanggung jawab atas akses data dan aturan data. Selain create() dan findByUsername() sesuai modul, ditambahkan pemeriksaan username dan email yang sudah terdaftar agar pelanggaran UNIQUE menghasilkan kode 409, bukan error 500 dari database. Validasi email (tugas mandiri 1) juga berada di Model ini.

```
<?php
class User {
    private PDO $db;

    public function __construct() {
        $this->db = Database::getConnection();
    }

    // Tugas Praktik Mandiri 1: validasi format email pada lapisan Model.
    // Dipanggil sebelum data disimpan; melempar InvalidArgumentException jika
tidak valid.
    private function validateEmail(string $email): void {
        if (filter_var($email, FILTER_VALIDATE_EMAIL) === false) {
            throw new InvalidArgumentException('Format email tidak valid');
        }
```

```
    }

    public function existsByUsername(string $username): bool {
        $stmt = $this->db->prepare('SELECT COUNT(*) FROM users WHERE username
= :u');
        $stmt->execute([':u' => $username]);
        return (int) $stmt->fetchColumn() > 0;
    }

    public function existsByEmail(string $email): bool {
        $stmt = $this->db->prepare('SELECT COUNT(*) FROM users WHERE email =
:e');
        $stmt->execute([':e' => $email]);
        return (int) $stmt->fetchColumn() > 0;
    }

    public function create(array $data): string {
        $this->validateEmail($data['email']);

        // Username/email UNIQUE: cek dulu agar menghasilkan 409, bukan error
500 dari database
        if ($this->existsByUsername($data['username'])) {
            throw new DomainException('Username sudah terdaftar');
        }
        if ($this->existsByEmail($data['email'])) {
            throw new DomainException('Email sudah terdaftar');
        }

        $stmt = $this->db->prepare(
            'INSERT INTO users (username, email, password) VALUES (:u, :e,
:p)'
        );
        $stmt->execute([':u' => $data['username'], ':e' => $data['email'],
':p' => $data['password']]);
        return $this->db->lastInsertId();
    }

    public function findByUsername(string $username) {
        $stmt = $this->db->prepare('SELECT * FROM users WHERE username = :u');
        $stmt->execute([':u' => $username]);
        return $stmt->fetch();
    }
}
```

e. Langkah 4 - AuthController

AuthController adalah Controller pertama yang bukan CRUD standar karena hanya berisi register(). Kata sandi di-hash sebelum dikirim ke Model, dan hash tidak dikembalikan dalam response. Exception dari Model diterjemahkan menjadi kode status: InvalidArgumentException menjadi 422 dan DomainException menjadi 409.

```php
<?php
// Controller pertama yang bukan CRUD standar: hanya aksi register().
// Aksi login() ditambahkan pada Acara 20.
class AuthController {
    private User $model;

    public function __construct() {
        $this->model = new User();
    }

    public function register(): void {
        $data = json_decode(file_get_contents('php://input'), true);

        $errors = [];
        foreach (['username', 'email', 'password'] as $field) {
            if (!is_array($data) || empty($data[$field]) ||
!is_string($data[$field])) {
                $errors[$field] = "Field $field wajib diisi";
            }
        }
        if ($errors) {
            Response::json(400, false, 'Username, email, dan password wajib
diisi', null, $errors);
        }

        // Kata sandi tidak pernah disimpan sebagai teks biasa
        $data['password'] = password_hash($data['password'],
PASSWORD_DEFAULT);

        try {
            $id = $this->model->create($data);
        } catch (InvalidArgumentException $e) {
            Response::json(422, false, $e->getMessage(), null, ['email' => $e-
>getMessage()]);
        } catch (DomainException $e) {
            $field = str_contains($e->getMessage(), 'Email') ? 'email' :
'username';
            Response::json(409, false, $e->getMessage(), null, [$field => $e-
>getMessage()]);
        }

        // Hash password tidak ikut dikembalikan ke klien
```

```
        Response::json(201, true, 'Registrasi berhasil', ['id' => $id]);
    }
}
```

f. Langkah 5 - Resource register pada Router

Karena register bukan resource CRUD, Router menanganinya lebih dulu sebelum blok switch. Request selain POST ke /register dijawab 405.

```
    private array $map = [
        'mahasiswa' => 'MahasiswaController',
        'buku'      => 'BukuController', // Tugas Praktik Mandiri 1
        'register'  => 'AuthController',  // Acara 19: autentikasi
    ];
...
        if ($resource === 'register') {
            if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
                Response::json(405, false, 'Method tidak diizinkan');
            }
            $controller->register();
            return;
        }
```

g. Langkah 6 - Pengujian dan Dokumentasi pada Collection

Request POST {{base_url}}/register ditambahkan ke Collection, lengkap dengan skrip uji sederhana (pm.test) yang memeriksa kode status dan format JSON. Contoh response sukses (201) dan gagal (400, 422, 409, 405) disimpan sebagai Saved Example pada masing-masing request sebagai dokumentasi awal.

2. Tugas Praktik Mandiri

a) ***Validasi email dengan filter_var()***

Metode validateEmail() pada Model User memanggil filter_var($email, FILTER_VALIDATE_EMAIL). Bila format salah, Model melempar InvalidArgumentException sebelum data disimpan, dan AuthController mengubahnya menjadi response 422 dengan field errors.email (Gambar 6).

b) ***Environment Staging***

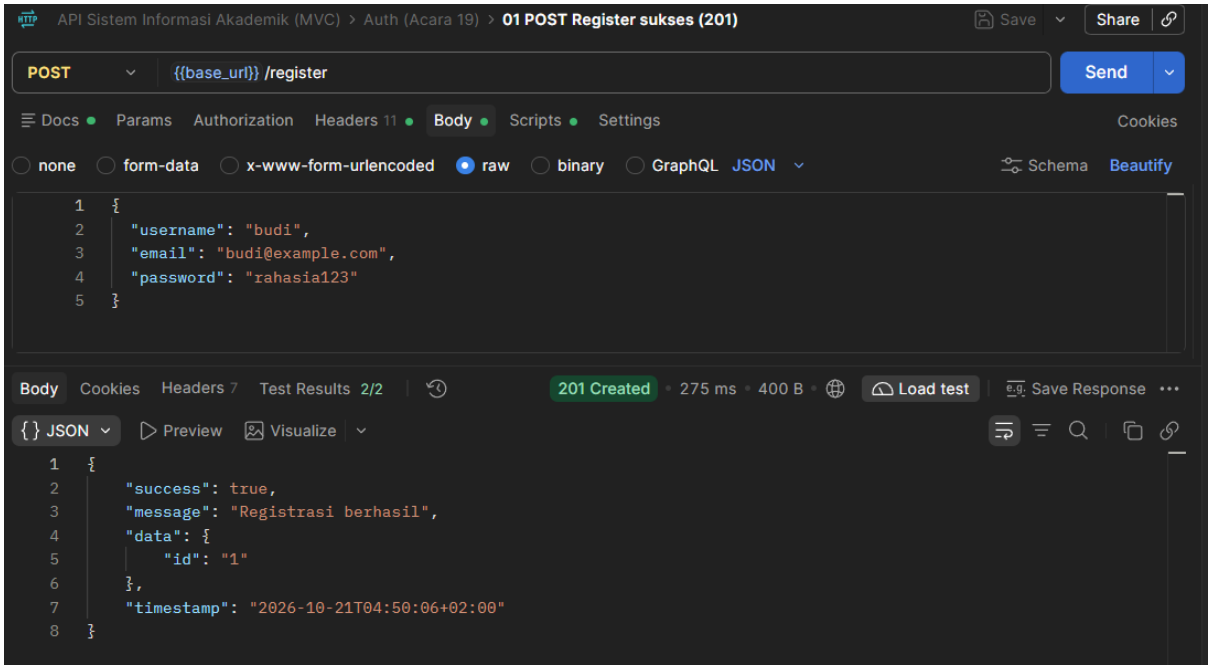
Dibuat Environment kedua bernama Staging dengan base_url berbeda (http://localhost:8080) untuk mensimulasikan server lain. Collection tidak perlu diubah; cukup mengganti Environment yang aktif (Gambar 1). Server Staging dijalankan dengan perintah php -S localhost:8080 -t public public/index.php dari folder proyek.

3. Pengujian API

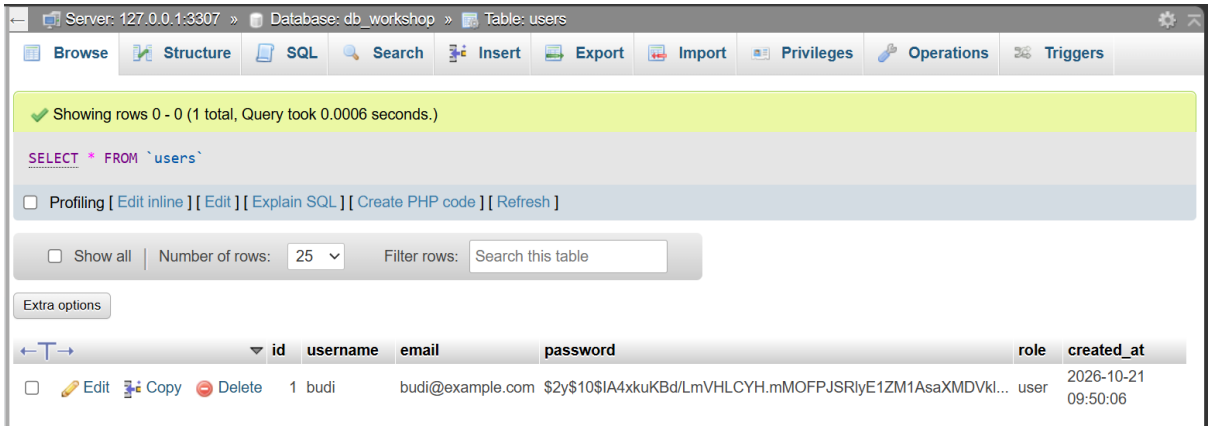
Pengujian dijalankan terhadap Environment Staging (http://localhost:8080) menggunakan curl. Request, header, dan body yang dikirim identik dengan request pada Collection Postman, sehingga hasilnya dapat diulang dari Postman.

Tabel 3. Hasil pengujian

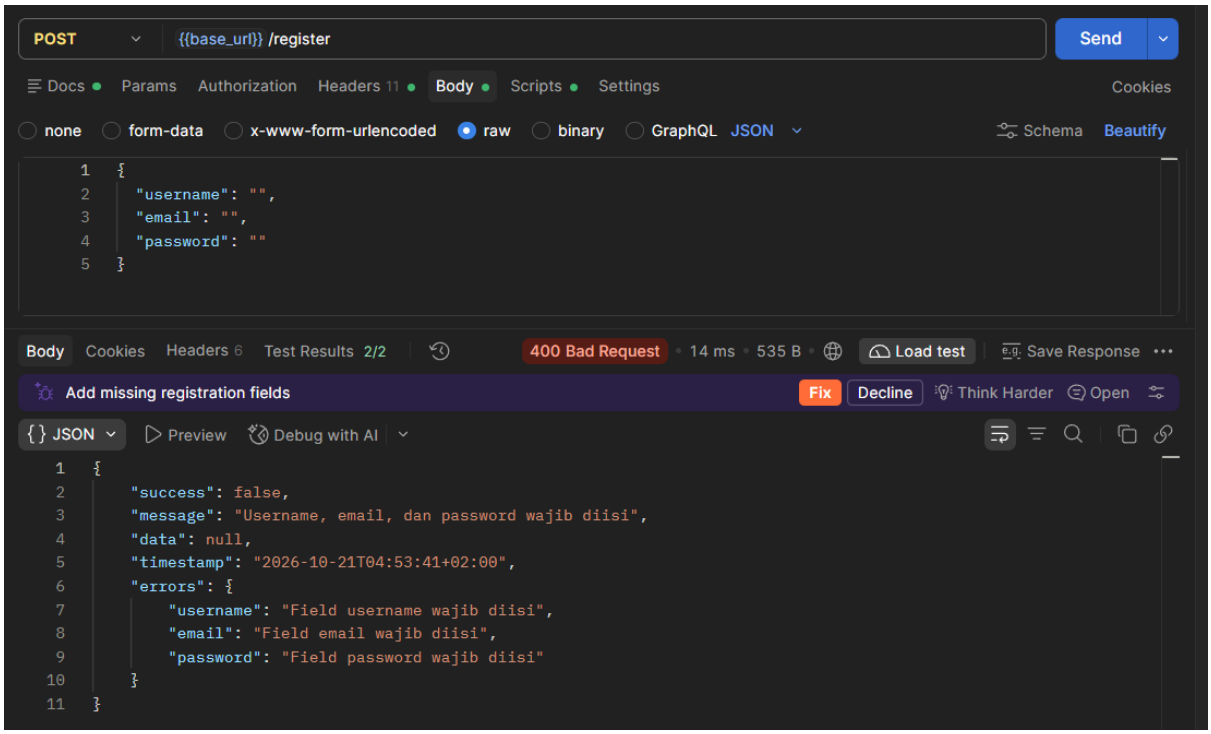
No	Skenario	Request	Status	Gambar
1	Struktur tabel users	-	-	2
2	Register data valid	POST /register	201 Created	3
3	Password tersimpan sebagai hash	SELECT users	Hash 60 karakter	4
4	Field kosong	POST /register	400 Bad Request	5
5	Email tidak valid	POST /register	422 Unprocessable Entity	6
6	Username sudah terdaftar	POST /register	409 Conflict	7
7	Email sudah terdaftar	POST /register	409 Conflict	8
8	Method salah	GET /register	405 Method Not Allowed	9
9	Endpoint mahasiswa tetap berfungsi	GET /mahasiswa	200 OK	10



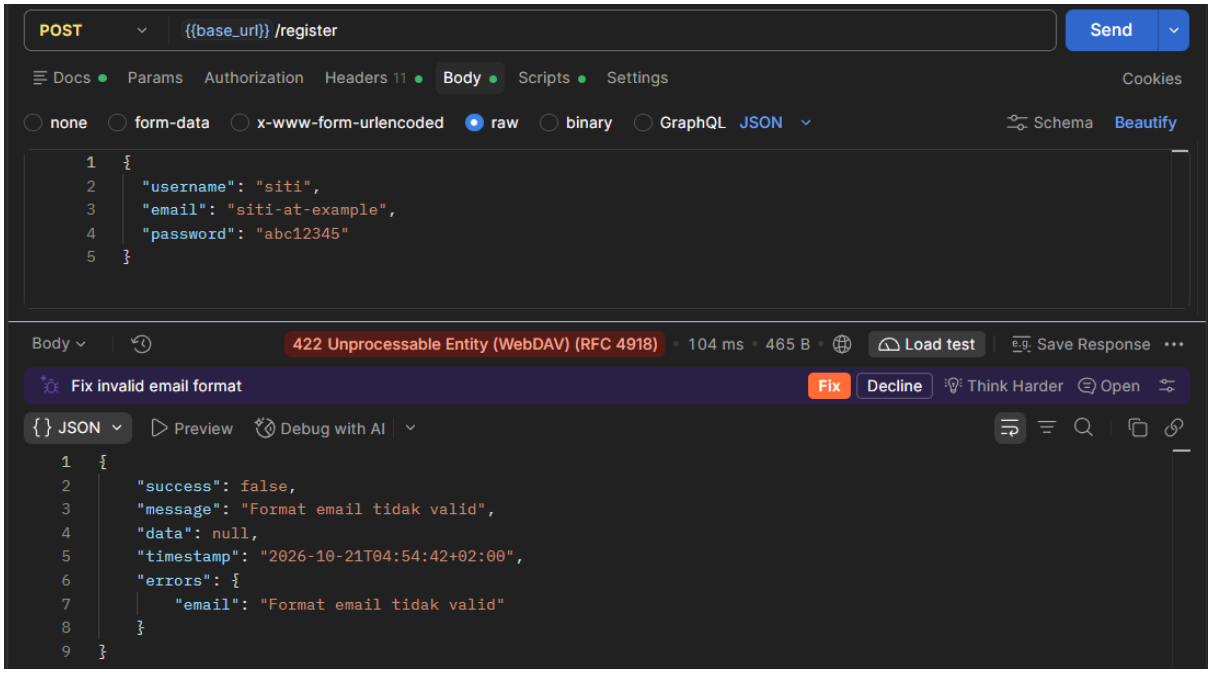
Gambar 3. POST /register data valid: 201 Created



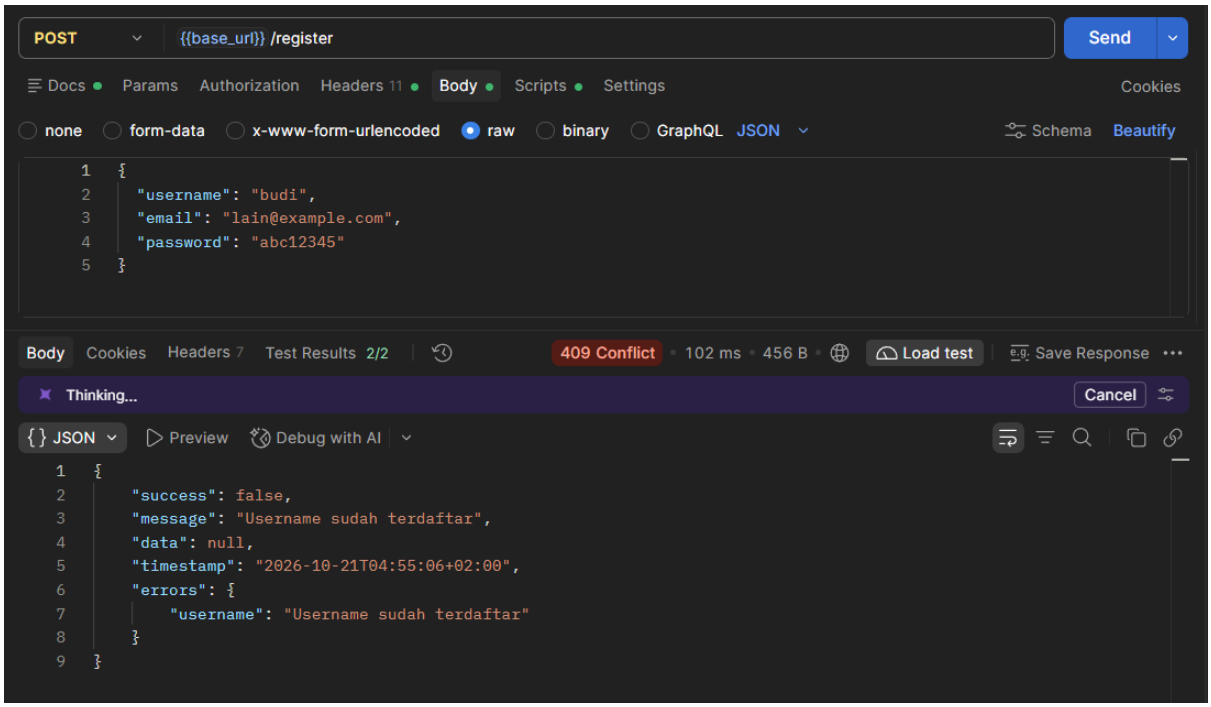
Gambar 4. Password tersimpan sebagai hash bcrypt, bukan teks biasa



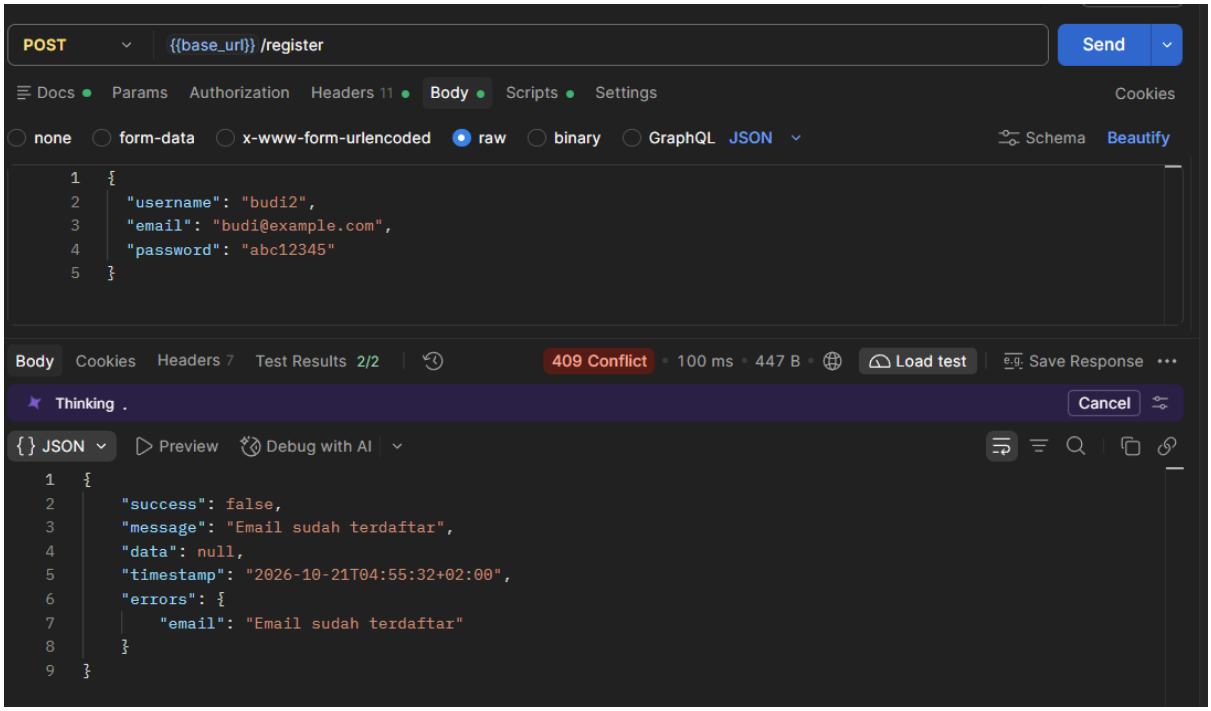
Gambar 5. POST /register field kosong: 400 Bad Request disertai errors



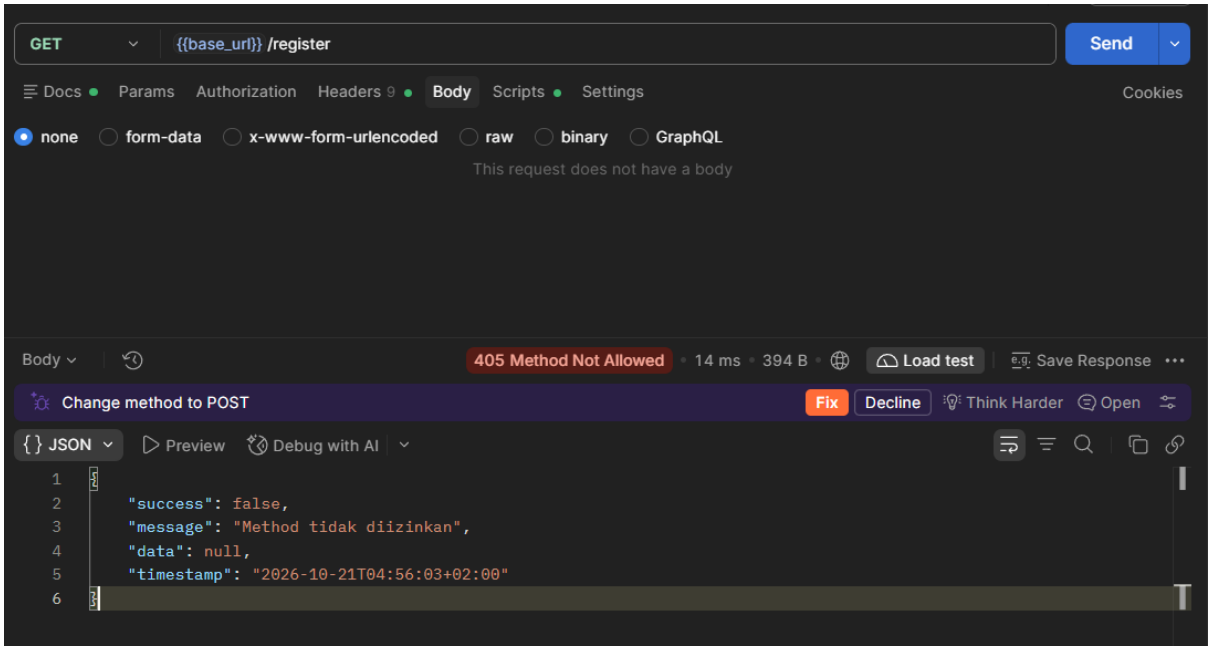
Gambar 6. POST /register email tidak valid: 422 Unprocessable Entity



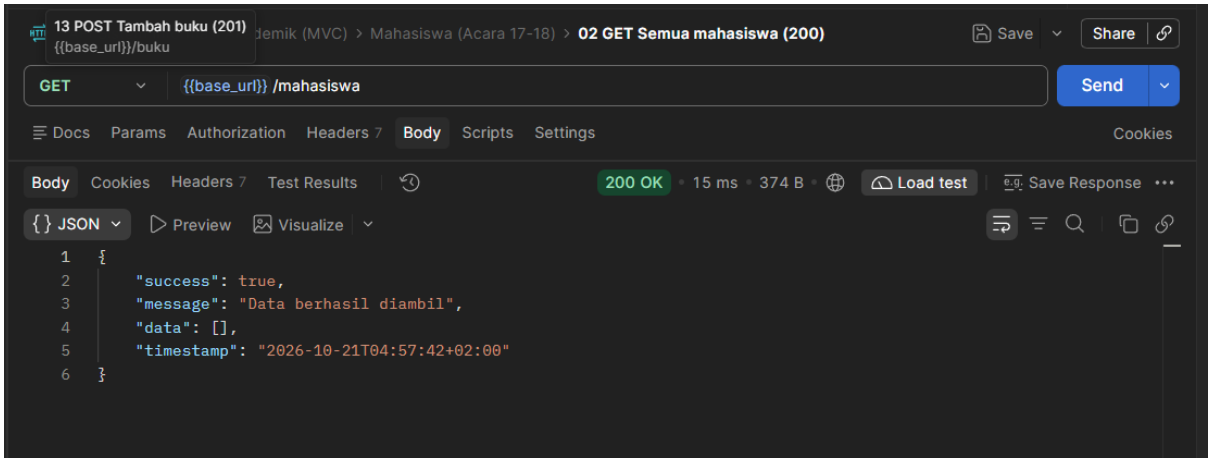
Gambar 7. POST /register username duplikat: 409 Conflict



Gambar 8. POST /register email duplikat: 409 Conflict



Gambar 9. GET /register: 405 Method Not Allowed



Gambar 10. GET /mahasiswa tetap 200 OK setelah perubahan

4. Pembahasan

Kata sandi "rahasia123" tersimpan sebagai hash bcrypt sepanjang 60 karakter (diawali $2y$), sehingga isi database tidak membocorkan kata sandi asli. Kolom password dibuat VARCHAR(255) karena PASSWORD_DEFAULT dapat berganti algoritma di versi PHP mendatang dengan hash yang lebih panjang. Response register hanya mengembalikan id, tidak pernah hash.

Pembagian tanggung jawab tetap konsisten dengan Acara 18: Controller memeriksa kelengkapan input (400), Model memeriksa aturan data seperti format email (422) dan duplikasi (409), dan Router menangani method yang tidak sesuai (405). Pemeriksaan duplikasi dilakukan di Model sebelum INSERT agar klien menerima 409 yang jelas, bukan 500. Pemeriksaan ini tidak sepenuhnya atomik, sehingga jika dua request identik tiba bersamaan, batas UNIQUE pada tabel tetap menjadi pengaman terakhir. Aksi login() dan pemakaian password_verify() baru dibuat pada Acara 20.

KESIMPULAN

1. Postman Collection "API Sistem Informasi Akademik (MVC)" berhasil dibuat dengan Environment Local dan Staging sehingga alamat server dapat diganti tanpa mengubah request.

2. Tabel users, Model User, AuthController, dan resource register pada Router berhasil dibuat; kata sandi disimpan sebagai hash memakai password_hash().

3. Pengujian berhasil untuk kode 201, 400, 405, 409, dan 422, dan endpoint mahasiswa tetap berfungsi.

4. Tugas mandiri selesai: validasi email dengan filter_var() pada Model User dan Environment Staging.

DAFTAR PUSTAKA

Pratama, R. A., & Hasanah, Q. (2026). Buku Kerja Praktik Mahasiswa (BKPM) Workshop Sistem Informasi Web Server (TIF330805). Program Studi D4 Teknik Informatika PSDKU Nganjuk, Jurusan Teknologi Informasi, Politeknik Negeri Jember.